

COMPLETE AI ENGINEER FRESHER ROADMAP

Absolute Beginner → Job-Ready → Production-Ready

A structured curriculum for Python, ML, Deep Learning, NLP, Computer Vision, Transformers, GenAI, LLMs, RAG, Agents, Backend, Docker, Cloud, MLOps, LLMOps, Security, Evaluation and AI System Design.

Priority	Meaning
☐ MUST LEARN	Core skills expected in most AI Engineer fresher roles
☐ SHOULD LEARN	High-value skills that improve job readiness
☐ GOOD TO KNOW	Useful specialization or production depth
◦ OPTIONAL	Learn later when a role/project requires it

Recommended primary stack: Python • SQL • Git • Linux • NumPy/Pandas • scikit-learn • PyTorch • Transformers • FastAPI • PostgreSQL • Redis • FAISS/Qdrant • LangGraph/LangChain or LlamaIndex • Docker • MLflow • one cloud (AWS recommended).

Important: This roadmap intentionally avoids treating every technology as equally important. Your goal as a fresher is to understand fundamentals, build several complete systems, deploy at least one, and explain your engineering decisions in interviews.

Table of Contents

- 1. Complete AI Engineer Fresher Roadmap
- 2. Prerequisite / Dependency Map
- 3. Concepts to Learn
- 4. Libraries, Frameworks and Tools
- 5. Architecture Patterns
- 6. Python for AI Engineers
- 7. Computer Science Fundamentals
- 8. SQL, Databases and Data Engineering
- 9. Machine Learning
- 10. Deep Learning
- 11. NLP and Computer Vision
- 12. Transformers
- 13. Generative AI and LLM Engineering
- 14. Embeddings and Vector Databases
- 15. RAG Engineering
- 16. Agentic AI
- 17. Backend Development
- 18. MLOps and LLMOps
- 19. Docker, Cloud and Kubernetes
- 20. AI Security and Evaluation
- 21. AI System Design
- 22. Project Roadmap
- 23. Six-Month Roadmap
- 24. Nine-Month Roadmap
- 25. Daily Routine
- 26. Interview Roadmap
- 27. Job Description Skill Mapping
- 28. 120-Skill Checklist
- 29. Final Job-Ready Test

1. 📁 Complete AI Engineer Fresher Roadmap

Recommended sequence

Programming → CS basics → SQL/Data → ML → Deep Learning → NLP/CV basics → Transformers → GenAI/LLMs → Embeddings → Vector Search → RAG → Agents → FastAPI/Backend → Docker → Cloud → MLOps/LLMOps → Security/Evaluation → AI System Design → Production Projects → Interviews.

Stage	Priority	Why it comes here	Job-ready outcome
Python + Git + Linux	🔴	Foundation for nearly every AI workflow	Write clean scripts, packages and CLIs
CS + SQL + Data	🔴	AI systems are software + data systems	Query data, debug services, reason about performance
ML	🔴	Teaches modeling, evaluation and failure modes	Train, validate and explain classical models
Deep Learning	🔴	Foundation for modern NLP/CV/LLMs	Train/fine-tune neural networks with PyTorch
NLP/CV basics	🔴	Gives modality understanding before foundation models	Build baseline text/image systems
Transformers	🔴	Core architecture behind modern LLMs/VLMs	Explain attention, inference and model families
GenAI + LLM APIs	🔴	Directly useful in current AI applications	Build reliable LLM features
Embeddings + Vector DB	🔴	Core building blocks for semantic search/RAG	Implement retrieval and metadata filtering
RAG	🔴	Common enterprise GenAI pattern	Build grounded document QA
Agents	🔴	Increasingly common but less predictable	Build tool-using workflows with state/evals
FastAPI + APIs	🔴	Turns models into usable services	Serve ML/LLM systems
Docker	🔴	Reproducible deployment	Containerize and run locally/cloud
Cloud	🔴	Production deployment and infrastructure	Deploy a small AI service
MLOps/LLMOps	🔴	Makes systems observable and maintainable	Track versions, evals, metrics and releases
System design + security	🔴	Separates demo projects from production thinking	Design scalable, safe AI architectures

Do NOT learn yet: training an LLM from scratch, advanced Kubernetes administration, distributed GPU training, every cloud provider, every agent framework, every vector database, advanced Spark internals, service meshes, custom CUDA kernels, research-level mathematical proofs.

2. 🗺 Prerequisite / Dependency Map

You want to learn	You should already know	Minimum depth
scikit-learn ML	Python, NumPy, Pandas, statistics	Comfortable
PyTorch	Python, arrays/tensors, calculus intuition	Basic math
Transformers	DL, neural nets, attention concept	Conceptual + coding
LLM APIs	Python, HTTP/JSON, prompts	Practical
Embeddings	Vectors, similarity, Python	Conceptual
RAG	LLMs, embeddings, vector search	Practical
Agents	LLM APIs, tools, structured outputs, state	Practical
FastAPI	Python, HTTP, JSON, REST	Practical
Docker	Linux/CLI, application structure	Practical
Cloud	Linux, HTTP, Docker, networking basics	Practical
MLOps	Git, ML lifecycle, Docker, basic cloud	Practical
AI system design	Backend, databases, queues, caching, ML/LLM	Integrated

Rule: Never jump to a framework because a tutorial uses it. First understand the underlying concept, then use the framework to reduce implementation effort.

3. 📌 Concepts to Learn

The following is the conceptual spine of the curriculum. Concepts are more durable than specific libraries.

Programming

- Variables/types
- Control flow
- Functions
- Data structures
- OOP
- Exceptions
- Modules/packages
- Typing
- Async I/O
- Testing
- Logging

Data

- Tabular data
- Missing values
- Outliers
- Encoding
- Scaling
- Sampling
- Train/validation/test
- Data leakage
- Data contracts

ML

- Features/labels
- Loss/objective
- Bias/variance
- Regularization
- Cross-validation
- Class imbalance
- Calibration
- Thresholding
- Feature engineering

DL

- Tensors
- Forward/backprop
- Optimizers
- Activations

- Normalization
- Regularization
- CNN/RNN intuition
- Attention
- Fine-tuning

LLM

- Tokens
- Context window
- Attention
- Inference
- Temperature
- Top-p
- Structured output
- Tool calling
- Streaming
- Caching

Retrieval

- Embeddings
- Similarity
- ANN indexes
- Metadata filters
- Hybrid retrieval
- Reranking
- Chunking
- Query transformation

Agents

- Tools
- State
- Memory
- Planning
- Routing
- Reflection
- Human approval
- Retries
- Guardrails
- Observability

Production

- API design
- Auth

- Containers
- CI/CD
- Secrets
- Metrics
- Tracing
- Drift
- Cost
- Rate limiting
- Failure recovery

4. 📦 Libraries & Frameworks

Recommended stack first; alternatives are included so you understand the ecosystem without trying to master everything.

Area	Primary	Alternatives	Priority	Use when
Numerical	NumPy	—	🔴	Arrays, vectorized computation
Data	Pandas	Polars	🔴 / 🟡	Data cleaning and feature preparation
ML	scikit-learn	XGBoost / LightGBM / CatBoost	🔴	Classical ML; boosting for tabular performance
DL	PyTorch	TensorFlow/Keras	🔴 / 🟡	Training/fine-tuning neural models
LLM models	Hugging Face Transformers	vLLM, llama.cpp, Ollama	🔴 / 🟡	Models, tokenizers and inference
LLM apps	Provider SDKs	OpenAI-compatible SDKs	🔴	Production LLM APIs
RAG	LlamaIndex or LangChain	Haystack	🔴	Orchestration; choose one first
Agents	LangGraph	OpenAI Agents SDK, CrewAI	🔴	Stateful agent workflows
API	FastAPI	Flask, Django/DRF	🔴	AI inference/backend APIs
Vector search	FAISS	Qdrant, Pinecone, Weaviate	🔴 / 🟡	Semantic retrieval
Database	PostgreSQL	MySQL, SQLite	🔴	Application data and metadata
Cache	Redis	—	🔴	Caching, rate limits, ephemeral state
Tracking	MLflow	Weights & Biases	🔴	Experiments/models/evals
Containers	Docker	—	🔴	Reproducible deployment
Workflow	Airflow	Prefect, Dagster	🔴	Scheduled pipelines

Framework principle: Learn one primary tool deeply enough to build and debug a project. Learn competing tools at comparison level. Add another only when a job/project requires it.

5. 🛠️ Tools

Category	Tools to recognize	Fresher priority
Code	VS Code, Jupyter, Git, GitHub	☐
Environment	venv/uv/Poetry, pip, .env	☐
Linux	bash, grep, curl, ssh, ps, top, chmod	☐
API testing	curl, Postman/Insomnia	☐
Data	DBeaver/pgAdmin, notebooks	☐
Observability	OpenTelemetry, Prometheus/Grafana concepts	☐
LLM tracing/evals	LangSmith, Arize/Phoenix, provider eval tools	☐
CI/CD	GitHub Actions	☐
Cloud	AWS console + CLI	☐
Containers	Docker CLI, Compose	☐
Orchestration	Kubernetes, Helm	☐

6. 📁 Architecture Patterns

A fresher should be able to draw these systems on a whiteboard and explain the data flow, failure modes and trade-offs.

ML prediction API

Client → FastAPI → validation → preprocessing → model → prediction → PostgreSQL/logging → metrics

RAG chatbot

User → API → query preprocessing → embedding → vector/hybrid retrieval → reranker → context builder → LLM → citations → evaluation/trace

AI Agent

User → API → agent state → LLM planner/router → tool call → tool result → validation/guardrail → next step → final response → trace/evaluation

Production AI platform

Frontend → API gateway → auth/rate limit → application service → model/LLM gateway → cache → DB/vector DB → queue/workers → observability → CI/CD → cloud runtime

Key production idea: model quality is only one part of the system. Reliability, data quality, latency, security, cost, observability and rollback strategy matter too.

7. Python for AI Engineers

What: Python is the main glue language for data, ML, APIs, automation and AI orchestration. Why: most AI tooling exposes Python interfaces. When: throughout the roadmap.

📌 MUST LEARN

- Syntax, variables, primitive types, input/output, operators, conditions and loops.
- Strings: indexing, slicing, formatting, split/join, replace, find, startswith/endswith, strip, case conversion.
- Lists: indexing/slicing, append, extend, insert, remove, pop, clear, sort, reverse, copy; list comprehensions.
- Tuples, sets and dictionaries: creation, indexing/lookup, update, membership, union/intersection/difference, get/items/keys/values.
- Functions, scope, return values, default/keyword arguments, *args, **kwargs, lambda, map/filter/reduce.
- Exceptions, modules/packages, imports, virtual environments, dependency management, file I/O and JSON.
- OOP: class/object, inheritance, composition, polymorphism, encapsulation, abstract interfaces.
- Iterators, generators, decorators, context managers, dataclasses and type hints.
- Testing with pytest concepts; logging; debugging; clean project structure.
- Async programming basics for I/O-bound AI APIs.

📌 SHOULD LEARN

- Threading vs multiprocessing
- Queues
- Profiling
- Packaging
- CLI tools
- Serialization trade-offs
- Performance/vectorization

Core AI stack

Library	Purpose	Learn to do	Priority
NumPy	Numerical arrays	reshape, broadcasting, vectorization, linear algebra basics	📌
Pandas	Tabular data	load, clean, join, group, aggregate, transform	📌
Polars	Fast dataframe engine	lazy/dataframe concepts	📌
Matplotlib	Visualization	plots for EDA and debugging	📌
Jupyter	Interactive development	experiments, analysis and reproducibility	📌

Python interview target: You should be able to write a clean 50–150 line solution without copying syntax, explain mutable vs immutable objects, identify $O(n)$ vs $O(1)$ operations, debug exceptions, and build a small API or data pipeline.

8. 📦 Computer Science Fundamentals

Topic	Fresher depth	Why
Arrays/lists, hash maps, stacks, queues	📖	Coding interviews + practical code
Trees/graphs	📖	Interview fundamentals and reasoning
Sorting/searching	📖	Complexity and coding basics
Big-O	📖	Performance reasoning
OOP/design principles	📖	Maintainable AI services
Processes/threads	📖	Concurrency and serving
Memory/CPU basics	📖	Model/data performance
HTTP/HTTPS	📖	API and LLM integration
REST/JSON/status codes	📖	Backend integration
DNS/TCP basics	📖	Cloud/debugging
Linux CLI	📖	Servers and Docker
Distributed systems internals	📖	Later system-design depth

9. 📦 SQL, Databases & Data Engineering

SQL is a must for AI engineers because model inputs, labels, user events, evaluation datasets and application state frequently live in databases.

SQL curriculum

- SELECT/WHERE/ORDER BY
- GROUP BY/HAVING/aggregations
- INNER/LEFT/RIGHT joins
- Subqueries and CTEs
- Window functions
- CASE/NULL handling
- Indexes and query plans
- Transactions/ACID
- Constraints
- Normalization
- Upserts and pagination

Database priorities

Technology	Use	Priority
PostgreSQL	Primary relational database; production-friendly	🔴
SQLite	Local prototypes/tests	🔴
MySQL	Common enterprise relational DB	🔴
MongoDB	Document-oriented data	🔴
Redis	Cache/rate limit/session/queues	🔴

Data engineering minimum

- Ingestion → validation → cleaning → transformation → storage
- ETL vs ELT
- Batch vs streaming
- Data quality checks
- Data lineage and schema awareness
- Basic warehouse/lake concepts

Tool	Priority	What to know
Pandas/Polars	🔴	Transform datasets
Airflow	🔴	DAGs, schedules, retries, dependencies
Spark	🔴	Distributed processing conceptually
Kafka	🔴	Events/partitions/consumer groups conceptually

10. 📦 Machine Learning

What: algorithms that learn patterns from data. Why: many AI jobs still include tabular prediction, ranking, anomaly detection or classical ML components.

Fundamentals

- Training/validation/testing
- Features, labels, parameters, hyperparameters
- Loss functions
- Generalization
- Bias/variance
- Data leakage
- Feature leakage
- Class imbalance

Algorithms

Category	Algorithms	Priority
Regression	Linear, Ridge, Lasso, Elastic Net	🔴
Classification	Logistic regression, KNN, Naive Bayes	🔴
Trees	Decision tree, Random Forest	🔴
Boosting	Gradient Boosting, XGBoost, LightGBM, CatBoost	🔴
Margin	SVM	🔴
Clustering	K-Means, hierarchical, DBSCAN	🔴
Dimensionality	PCA	🔴
Anomaly	Isolation Forest and related methods	🔴

Techniques

- Encoding: one-hot/ordinal/target encoding with leakage awareness
- Scaling: standard/min-max/robust
- Imputation
- Feature selection
- Cross-validation
- Grid/random/Bayesian tuning
- Regularization
- Ensembling
- Calibration and threshold selection

Evaluation

Problem	Metrics
Classification	Confusion matrix, accuracy, precision, recall, F1, ROC-AUC, PR-AUC
Regression	MAE, MSE, RMSE, R ²
Real-world	Business cost, latency, calibration, coverage, false-positive/false-negative cost

Algorithm selection heuristic: start with a simple baseline → inspect data → try strong tree boosting for tabular data → compare using cross-validation → tune only after a sound validation setup → select based on business metric and operational constraints, not accuracy alone.

11. 📦 Deep Learning

📌 Prioritize PyTorch. Learn TensorFlow/Keras mainly to understand the ecosystem or satisfy a job requirement. PyTorch is a strong default for modern research-to-production workflows and Hugging Face integration.

Core concepts

- Perceptron and neural network
- Tensors
- Forward pass
- Backpropagation
- Gradient descent
- Activations: ReLU, sigmoid, tanh, softmax
- Loss functions
- Optimizers: SGD, Adam/AdamW
- Learning rate schedules
- Batch size/epoch
- Weight initialization
- Dropout
- Batch normalization
- Early stopping

Architectures

Architecture	Know it for	Priority
ANN/MLP	Basic neural networks	📌
CNN	Image understanding	📌
RNN/LSTM/GRU	Historical sequence modeling and interviews	📌
Autoencoder/VAE	Representation/generative concepts	📌
GAN	Generative model history	📌
Transformer	Modern language/multimodal foundation	📌

Practical target

- Build a PyTorch training loop from scratch
- Use Dataset/DataLoader
- Track train/validation loss
- Save/load checkpoints
- Use GPU when available
- Fine-tune a pretrained model
- Diagnose overfitting and learning-rate problems

12. 📦 NLP + 📦 Computer Vision

Modern AI engineers do not need every classical NLP/CV technique, but they need enough fundamentals to understand model inputs, outputs and failure modes.

NLP

- Tokenization
- Normalization
- Stopwords/stemming/lemmatization — understand, don't overinvest
- POS/NER
- Text classification
- Similarity
- TF-IDF
- Word2Vec/GloVe/FastText concepts
- Seq2Seq
- Attention

CV

- Pixels/channels/color spaces
- Resize/normalize/augment
- Classification
- Object detection
- Segmentation
- OCR
- Face detection/recognition concepts
- Pose estimation
- Tracking

Tool/model	Purpose	General AI priority
OpenCV	Image/video processing	📦
Ultralytics YOLO	Object detection/vision workflows	📦
Tesseract/EasyOCR	OCR baselines	📦
Transformers	Modern NLP/CV/VLM models	📦
CLIP/VLMs	Vision-language understanding	📦

Generalist rule: learn one end-to-end CV project and one NLP project. Do not become a CV/NLP specialist unless the target job requires it.

13. 📦 Transformers

What: a neural architecture built around attention. Why: it powers most modern LLM and many multimodal systems.

Must understand

- Encoder-only, decoder-only, encoder-decoder
- Token embeddings
- Positional information
- Query/Key/Value
- Scaled dot-product attention
- Multi-head attention
- Feed-forward network
- Residual connections
- Layer normalization
- Causal masking
- Context window
- Autoregressive generation
- KV cache
- Inference

Model families to recognize

Family	Architecture/role	What to know
BERT	Encoder-only	Understanding/classification/embeddings history
GPT-style	Decoder-only	Autoregressive generation
T5/BART	Encoder-decoder	Seq2seq/generation
Llama/Mistral/Qwen/Gemini/DeepSeek	Modern foundation families	Capabilities, licensing/access, inference trade-offs

AI Engineer vs researcher: engineer = architecture intuition, tokenization, inference, context, fine-tuning and serving. Researcher = deeper optimization theory, scaling laws, architecture experiments and training infrastructure.

14. 📦 Generative AI & LLM Engineering

Lifecycle: User → Application → Prompt/context → LLM → tools/RAG → structured output → validation → evaluation → monitoring.

Prompting

- System/user/developer instruction roles
- Zero-shot/few-shot
- Task decomposition
- Structured outputs/JSON schema
- Prompt templates
- Context engineering
- Tool/function calling
- Output validation

Generation controls

- Temperature
- Top-p
- Maximum output tokens
- Stop sequences where supported
- Streaming
- Token counting
- Rate limits
- Retries/timeouts

Prompting vs RAG vs fine-tuning vs pretraining

Technique	Changes	Best use
Prompting	Instructions/context at inference	Behavior/task instructions
RAG	External knowledge at inference	Private/current/domain knowledge
Fine-tuning	Model parameters/adapters	Style, format, task behavior; domain adaptation where justified
Pretraining	Learn foundation model from huge corpora	Organizations with massive data/compute

Model selection

- Quality on your actual task
- Latency
- Context requirements
- Tool calling/structured output support
- Privacy/data handling
- Cost
- Rate limits
- Hosting/control
- Evaluation results

15. 📁 Embeddings & Vector Databases

Embeddings map text or other objects into vectors so semantically related items can be retrieved using similarity.

Similarity

- Cosine similarity
- Dot product
- Euclidean distance
- Dense vs sparse representations
- Normalization
- Query/document embeddings
- Multilingual embeddings

Vector database comparison

Technology	Type	Strength	Fresher priority
FAISS	Library	Local vector indexing; excellent learning/prototyping	📌
Chroma	Vector DB	Simple developer experience	📌
Qdrant	Vector DB	Filtering + production-oriented retrieval	📌
Pinecone	Managed vector DB	Managed scaling/operations	📌
Weaviate	Vector DB	Rich retrieval ecosystem	📌
Milvus	Vector DB	Large-scale vector search	📌
Elasticsearch/OpenSearch	Search engine	Keyword + hybrid/search platform	📌

Learn first: implement cosine similarity manually → use FAISS → learn metadata filtering/hybrid retrieval → then use one production-oriented vector DB such as Qdrant or a managed service when deploying.

16. 📦 RAG Engineering

RAG is not simply 'put documents in a vector DB'. It is a retrieval system plus context construction plus generation plus evaluation.

20-step RAG pipeline

- Ingest documents → parse → clean → normalize → identify metadata
- Chunk → embed → index
- Receive query → rewrite/transform if needed
- Retrieve using vector, keyword or hybrid search
- Filter metadata → rerank → select context
- Construct grounded prompt → generate answer
- Attach citations → validate output
- Trace retrieval/generation → evaluate → monitor

RAG patterns

Pattern	Purpose	Priority
Naive RAG	Baseline retrieval + generation	🔴
Hybrid RAG	Keyword + vector retrieval	🔴
Multi-query	Generate multiple retrieval perspectives	🔴
Parent-child	Retrieve small chunks, answer with larger context	🔴
Reranking	Improve relevance after initial retrieval	🔴
Corrective/Self-RAG	Use feedback/self-checking to improve retrieval	🔴
Graph RAG	Graph-structured knowledge retrieval	🔴
Agentic RAG	Agent decides retrieval/tool workflow	🔴

Framework priority

Learn one first: LangChain if you want broad orchestration and integrations, or LlamaIndex if your main focus is document/RAG applications. Learn the second only after you can implement RAG without hiding the core retrieval logic.

RAG quality checklist

- Chunking quality
- Embedding-model fit
- Top-k selection
- Recall@k
- Reranking
- Groundedness/faithfulness
- Citation correctness
- No-answer behavior
- Latency and cost

17. 🧠 Agentic AI

An agent is best treated as a controlled software workflow in which an LLM selects actions/tools or decisions using state and feedback.

Core components

- Tool definitions
- Structured inputs/outputs
- State
- Memory
- Planning
- Routing
- Retries/timeouts
- Human-in-the-loop
- Guardrails
- Tracing/evaluation

Patterns

Pattern	Use	Priority
Tool calling	LLM invokes deterministic functions	🔴
Router	Choose specialist path	🔴
ReAct-style	Reason/action loop	🔴
Plan-and-execute	Separate planning and execution	🔴
Supervisor	Coordinator controls workers	🔴
Reflection	Critique/revise workflow	🔴
Multi-agent	Multiple specialized agents	🔴

Frameworks

Framework	Primary purpose	Priority
LangGraph	Stateful, controllable agent workflows	🔴/🔴
LangChain	LLM orchestration/integrations	🔴
OpenAI Agents SDK	Agent/tool workflows around provider ecosystem	🔴
CrewAI	Role-based multi-agent workflows	🔴
AutoGen	Agent conversation/orchestration	🔴
Semantic Kernel	Enterprise-oriented orchestration	🔴
MCP	Standardized tool/context connectivity concept	🔴

Do not start with multi-agent. First build a deterministic tool-calling workflow, add state, add evaluation, then introduce autonomy only where it provides measurable value.

18. ⚡ Backend Development for AI

Priority: FastAPI. It fits Python AI services, async I/O, typed request/response models and modern API development.

Must know

- HTTP methods/status codes
- REST endpoints
- JSON/schema validation
- Path/query/body parameters
- Authentication vs authorization
- Middleware
- Exception handling
- Logging
- CORS basics
- Async/await
- Streaming responses
- WebSockets conceptually

Framework comparison

Framework	Strength	Fresher priority
FastAPI	Modern typed Python APIs; excellent for AI inference	☐
Flask	Minimal/simple Python web services	☐
Django	Full web platform with ORM/admin	☐
Django REST Framework	REST APIs inside Django	☐

Production API target

- /health and /ready endpoints
- Request validation
- Timeouts/retries
- Structured logging
- Metrics
- Authentication
- Rate limiting
- Model loading strategy
- Graceful shutdown
- Docker deployment

19. 📦 MLOps & LLMOps

MLOps makes ML reproducible and deployable. LLMOps extends the same discipline to prompts, model versions, datasets, traces, evaluations, costs and safety.

Area	Tools	Priority	What you need
Experiment tracking	MLflow / W&B	🔴	Log params, metrics, artifacts
Data versioning	DVC	🔴	Know what data produced a model
Model registry	MLflow	🔴	Version/stage models
Pipelines	Airflow	🔴	Schedule/retry dependencies
CI/CD	GitHub Actions	🔴	Test/build/deploy automatically
Containerization	Docker	🔴	Reproducible runtime
Monitoring	Prometheus/Grafana concepts	🔴	Latency, errors, resource metrics
LLM tracing/evals	LangSmith / Phoenix / provider tools	🔴	Trace prompts, retrieval and outputs
Observability	OpenTelemetry	🔴	Standardized traces/metrics

Production loop: Data → Train/Evaluate → Register → Package → Deploy → Monitor → Detect drift/failure → Retrain or rollback.

20. ☐ Docker, ☁ Cloud & Kubernetes

Docker

- Images vs containers
- Dockerfile
- layers/caching
- ports
- environment variables
- volumes
- networks
- Compose
- registries

Containerize four things

- FastAPI ML API
- PyTorch inference service
- RAG API + vector DB
- Agent API + Redis/PostgreSQL

Cloud choice

Recommended first cloud: AWS. Learn the concepts through one provider instead of trying to learn AWS + Azure + GCP simultaneously.

AWS service	What to learn	Priority
IAM	Users/roles/policies/least privilege	☐
S3	Object storage	☐
EC2	Virtual machines	☐
ECS	Container deployment	☐
RDS	Managed relational database	☐
Lambda	Serverless functions concept	☐
CloudWatch	Logs/metrics	☐
Bedrock	Managed foundation-model access	☐
SageMaker	Managed ML platform	☐
EKS	Managed Kubernetes	☐

Kubernetes: ☐ GOOD TO KNOW for most freshers. Know Pods, Deployments, Services, ConfigMaps, Secrets, Ingress and scaling conceptually. Go deeper only if the job explicitly expects Kubernetes.

21. 📁 AI Security & Evaluation

Security

- Prompt injection and indirect prompt injection
- Jailbreaking and unsafe instructions
- PII/data leakage
- Authentication/authorization
- Secrets management; never hard-code API keys
- RAG access control and tenant isolation
- Tool allowlists and least privilege
- Input/output validation
- Rate limiting and abuse prevention
- Dependency and container supply-chain hygiene

LLM evaluation

Dimension	What to measure
Correctness	Does the answer satisfy the task?
Relevance	Does it answer the actual question?
Groundedness	Is it supported by provided evidence?
Faithfulness	Does generated text remain consistent with source/context?
Safety	Does it avoid unsafe or prohibited behavior?
Latency	Time to first token and total response time
Cost	Input/output token or compute cost
Reliability	Error, timeout, retry and tool-failure rates

Evaluation tools to recognize

Provider evaluation APIs, MLflow evaluation capabilities, LangSmith, Arize/Phoenix and custom Python evaluation pipelines. Tool choice matters less than having a repeatable dataset, measurable criteria and regression tests.

22. 📁 AI System Design

For interviews, learn to move from requirements → architecture → data flow → model choice → API/database → scale → reliability → security → monitoring → cost.

System	Core architecture
Recommendation	Events → feature store/data pipeline → candidate generation → ranking → cache → API → feedback loop
Fraud detection	Transaction stream → feature computation → model/rules → decision API → alerts → monitoring
RAG enterprise chatbot	Auth → API → retrieval → rerank → LLM → citations → audit/trace
AI search	Ingest → index → hybrid retrieval → rerank → answer/search UI
Document AI	Upload → object storage → OCR/parser → extraction model/LLM → validation → DB
Voice AI	Audio stream → ASR → orchestration → LLM/tool → TTS → latency monitoring
Agent	API → state → planner/router → tools → validation → response → audit
LLM platform	Gateway → model routing → cache → provider/local inference → guardrails → tracing/cost

Every design should answer: What happens if the model is slow? unavailable? wrong? the DB is down? the vector index is stale? the user is unauthorized? the request is duplicated? costs spike?

23. 📁 Project Roadmap

Build fewer projects, but make them complete. Every serious project should have README, architecture diagram, tests, configuration, API docs, Dockerfile, evaluation results and a clear limitations section.

10 Beginner Projects

Project	Scope
1. Data Quality CLI	Validate CSV/JSON datasets; Python, Pandas, pytest
2. SQL Analytics API	PostgreSQL + FastAPI + SQL queries
3. Customer Churn Baseline	scikit-learn pipeline + metrics + error analysis
4. Fraud Classification Baseline	Class imbalance + PR-AUC + thresholding
5. House/Price Predictor API	Turn a basic model into a tested API
6. Image Classifier	PyTorch CNN + augmentation
7. OCR Pipeline	OpenCV + OCR + post-processing
8. Sentiment API	Text preprocessing + pretrained model
9. Semantic Search Demo	Embeddings + FAISS
10. LLM Structured Extractor	LLM + JSON schema + validation

10 Intermediate Projects

Project	Scope
1. Production ML API	FastAPI + Docker + PostgreSQL + logging
2. Document QA RAG	Parsing + chunking + embeddings + citations
3. Hybrid RAG	Keyword + vector + reranking
4. Resume/Job Matcher	Embeddings + ranking + evaluation
5. Invoice Intelligence	OCR + extraction + validation
6. Support Ticket Router	Classifier/LLM + structured routing
7. Knowledge Search	Metadata filters + access control
8. LLM Evaluation Harness	Dataset + scoring + regression tests
9. Tool-Calling Assistant	LLM + safe tools + structured outputs
10. Async AI API	Streaming + queues + Redis + background workers

10 Advanced Projects

Project	Scope
1. Agentic RAG	LangGraph-style stateful workflow + retrieval + tools
2. Multi-source Research Assistant	Search/retrieval + citations + evaluation
3. Document Intelligence Platform	Upload → OCR → extraction → human review
4. AI Search Engine	Hybrid retrieval + reranking + analytics
5. Model Router	Route requests by quality/latency/cost
6. LLM Gateway	Auth + rate limits + logging + provider abstraction
7. ML Monitoring System	Drift + data quality + model metrics
8. Batch Inference Pipeline	Object storage + workflow + model registry
9. Voice AI Prototype	ASR → LLM → tools → TTS with streaming

Project	Scope
10. Secure Agent Platform	Tool permissions + audit logs + approvals

Production-level + recruiter-impress projects

Project	Production features that make it stand out
Enterprise Document Intelligence	Multi-format ingestion, OCR, extraction, validation, citations, auth, async jobs, Docker, monitoring
Secure RAG Knowledge Assistant	Tenant-aware retrieval, ACL filters, hybrid search, reranking, eval suite, tracing
Agentic Operations Assistant	Tool allowlist, approvals, state machine, retries, audit logs, cost controls
AI Model/LLM Gateway	Provider abstraction, routing, fallbacks, rate limits, caching, cost and latency dashboards
End-to-End ML Platform	Data versioning, experiment tracking, model registry, CI/CD, API deployment, drift monitoring

Resume bullet template

Built [system] using [stack], processing [scale], achieving [measured result], with [deployment/evaluation/monitoring feature].

For each project, prepare 5 interview answers: architecture, why this model/tool, failure case, evaluation method, and what you would change at 10× traffic.

24. 📅 Six-Month Roadmap (3-4 hours/day)

Month	Learning focus	Project	Exit criteria
1	Python, Git, Linux, SQL, NumPy/Pandas	SQL Analytics + Data Quality CLI	Solve Python basics without copying; write joins/CTEs
2	ML + statistics + evaluation	Production-style tabular ML API	Train, validate, explain leakage and metrics
3	PyTorch + DL + NLP/CV basics	Image or text classifier	Write training loop and diagnose overfitting
4	Transformers + LLM APIs + embeddings	LLM structured extractor + semantic search	Explain tokens/attention/inference
5	RAG + FastAPI + Docker + evaluation	Evaluated RAG chatbot	Deploy locally with citations, tests and evals
6	Agents + cloud + MLOps/LLMOps + system design	Production-style AI assistant	Docker + cloud + monitoring + architecture interview

End-of-month rule

- Month 1: GitHub has clean Python/SQL work and one README-quality project.
- Month 2: You can compare models and explain metrics/leakage.
- Month 3: You can build and train a PyTorch model.
- Month 4: You can explain transformer inference and build an LLM application.
- Month 5: You can build/evaluate/deploy RAG.
- Month 6: You can discuss production architecture, security, monitoring and trade-offs.

25. 📅 Nine-Month Roadmap

Phase	Months	Goal
Foundation	1-2	Python, SQL, Git/Linux, ML fundamentals
Modeling	3-4	PyTorch, NLP/CV, Transformers, evaluation
GenAI	5-6	LLMs, embeddings, RAG, FastAPI, Docker
Production	7	Agents, security, observability, cloud
MLOps/System Design	8	MLflow, CI/CD, architecture, scalability
Job Sprint	9	Portfolio polish, resume, mock interviews, applications

Weeks 33-36: Job sprint

- Refactor 2 strongest projects
- Add architecture diagrams and test coverage
- Add deployment link if feasible
- Write 5-7 strong resume bullets
- Practice Python/SQL daily
- Practice 2 AI system designs/week
- Run mock interviews
- Apply consistently to targeted roles

26. 📅 Daily Routine

Block	Time	Activity
Theory	45 min	One concept; take short notes
Coding	60 min	Python/SQL/ML problems
Project	75 min	Build, test, debug, document
Interview	20 min	Questions + explain aloud
Revision	20 min	Flashcards/summary/previous errors

Weekly rhythm: 5 days build/learn, 1 day project refactor + interview, 1 lighter day for review and planning. Every week produce a visible artifact: code, test, diagram, benchmark, README update or deployment.

27. 📅 Interview Roadmap

Area	Basic	Intermediate	Scenario/debugging
Python	Types, list/dict, functions	OOP, generators, decorators	Optimize slow code, debug exception
SQL	SELECT/JOIN	CTE/window/indexes	Slow query/data mismatch
ML	Algorithms/metrics	CV, leakage, tuning	Imbalanced fraud model
DL	NN/backprop	PyTorch training	Overfitting/GPU issue
Transformers	Attention	KV cache/context	Latency/context overflow
LLM	Prompt/API	Model selection	Cost/latency/quality trade-off
RAG	Embeddings/vector DB	Hybrid/rerank/evals	Low retrieval recall
Agents	Tools/function calling	State/guardrails	Tool loop failure
FastAPI	REST/JSON	Async/auth	Timeouts/race conditions
Docker	Image/container	Compose/network	Works locally, fails in container
Cloud	S3/IAM/compute	Deployment/monitoring	Permission or scaling issue
MLOps	Tracking/versioning	CI/CD/registry	Rollback/drift
System design	Components	Scale/trade-offs	DB/cache/queue/model outage

High-frequency scenario prompts

- Your RAG answers are fluent but frequently wrong. How do you debug it?
- Latency doubled after adding reranking. What would you measure and change?
- An LLM provider is down. How should your application behave?
- Your classifier has 99% accuracy but misses most fraud. What is wrong?
- A Dockerized API works locally but cannot connect to PostgreSQL. Debug it.
- How would you secure an agent that can send emails or modify records?
- Design an AI support system for 100 requests/second with auditability.

28. 📄 Job Description Skill Mapping

Role	Core skills	Depth	Proof
AI Engineer	Python, ML/DL, LLM, RAG, APIs, Docker	Broad + practical	2-3 end-to-end projects
ML Engineer	ML, PyTorch/sklearn, data, MLOps	Deeper modeling	ML deployment + monitoring
GenAI Engineer	LLMs, RAG, agents, evals, APIs	Deep GenAI	RAG/agent + eval/observability
Junior AI/ML Engineer	Python, ML basics, SQL, Git	Fundamental	Clean ML project + coding
AI Software Engineer	Python/backend, APIs, LLM integration	Backend-heavy	Production API + AI feature

Common overlap

Python + Git + SQL + ML fundamentals + API development + deployment basics + communication are the common denominator. Specialize after you have the shared foundation.

29. 120+ AI Engineer Skill Checklist

☐ Python syntax	☐ Python data types	☐ Strings	☐ Lists
☐ Tuples	☐ Sets	☐ Dictionaries	☐ Slicing/indexing
☐ Comprehensions	☐ Functions	☐ args/kwargs	☐ Lambda/map/filter
☐ Exceptions	☐ Modules/packages	☐ OOP	☐ Inheritance
☐ Composition	☐ Decorators	☐ Generators	☐ Context managers
☐ Type hints	☐ Dataclasses	☐ Async/await	☐ Threading vs multiprocessing
☐ Testing	☐ Logging	☐ Debugging	☐ Virtual environments
☐ Git basics	☐ GitHub	☐ Linux CLI	☐ HTTP
☐ REST	☐ JSON	☐ API status codes	☐ NumPy
☐ Pandas	☐ Polars	☐ Matplotlib	☐ Jupyter
☐ SQL SELECT	☐ SQL JOINS	☐ GROUP BY	☐ CTEs
☐ Window functions	☐ Subqueries	☐ Indexes	☐ Transactions
☐ Normalization	☐ PostgreSQL	☐ SQLite	☐ MongoDB basics
☐ Redis basics	☐ Data cleaning	☐ Data validation	☐ ETL/ELT
☐ Batch pipelines	☐ Streaming concepts	☐ Airflow concepts	☐ Data warehouses/lakes
☐ Train/validation/test	☐ Feature engineering	☐ Scaling	☐ Encoding
☐ Imputation	☐ Cross-validation	☐ Regularization	☐ Class imbalance
☐ Data leakage	☐ Bias/variance	☐ Linear regression	☐ Logistic regression
☐ KNN	☐ Naive Bayes	☐ Decision trees	☐ Random forest
☐ Gradient boosting	☐ XGBoost/LightGBM	☐ K-Means	☐ PCA
☐ Anomaly detection	☐ Classification metrics	☐ Regression metrics	☐ Threshold selection
☐ Calibration	☐ PyTorch tensors	☐ Training loop	☐ Backpropagation
☐ Optimizers	☐ Schedulers	☐ Activations	☐ Loss functions
☐ Dropout	☐ BatchNorm	☐ CNN	☐ RNN/LSTM concepts
☐ Transformer architecture	☐ Tokenization	☐ Embeddings	☐ Attention
☐ BERT concepts	☐ GPT concepts	☐ Seq2Seq concepts	☐ LLM API integration
☐ Prompt design	☐ Structured output	☐ Tool calling	☐ Streaming
☐ Context windows	☐ Model selection	☐ Temperature/top-p	☐ Token/cost awareness
☐ Inference latency	☐ Dense embeddings	☐ Cosine similarity	☐ FAISS
☐ Vector DB concepts	☐ Metadata filtering	☐ Hybrid retrieval	☐ Reranking
☐ Chunking	☐ Query transformation	☐ RAG evaluation	☐ Citations
☐ RAG security	☐ Agent state	☐ Agent tools	☐ Agent routing
☐ Agent retries	☐ Human-in-the-loop	☐ LangChain/LlamaIndex concepts	☐ LangGraph concepts
☐ MCP concept	☐ FastAPI	☐ Authentication	☐ Authorization
☐ Middleware	☐ Async APIs	☐ WebSockets/streaming	☐ Dockerfile
☐ Docker Compose	☐ Volumes/networks	☐ AWS IAM	☐ AWS S3
☐ AWS compute basics	☐ Cloud monitoring	☐ MLflow	☐ CI/CD
☐ Model registry	☐ Drift monitoring	☐ LLM tracing	☐ Prompt versioning

☐ Cost monitoring	☐ Rate limiting	☐ Prompt injection defense	☐ Secrets management
☐ System design	☐ Caching	☐ Queues	☐ Failure handling

Level key: Beginner = explain/use; Intermediate = build/debug; Job Ready = build + deploy + explain trade-offs; Advanced = optimize/design at scale.

30. ☐ “AM I READY FOR MY FIRST AI ENGINEER JOB?”

Complete these 50 tasks. Score each task 0, 1 or 2: 0 = cannot do; 1 = can do with documentation/help; 2 = can do independently and explain trade-offs.

1. ☐ Write a clean Python program using functions and collections.
2. ☐ Explain mutable vs immutable objects with examples.
3. ☐ Use comprehensions and dictionary operations confidently.
4. ☐ Create a small package with a virtual environment.
5. ☐ Write unit tests for a Python module.
6. ☐ Use Git branches, commits, pull/merge workflow and .gitignore.
7. ☐ Use Linux CLI to inspect files/processes and call an API with curl.
8. ☐ Write SQL joins, CTEs and window functions.
9. ☐ Design a normalized PostgreSQL schema.
10. ☐ Clean and validate a real dataset.
11. ☐ Build a scikit-learn preprocessing + model pipeline.
12. ☐ Explain train/validation/test and cross-validation.
13. ☐ Diagnose data leakage.
14. ☐ Handle class imbalance and select a useful metric.
15. ☐ Compare at least three ML models using a fair validation setup.
16. ☐ Explain bias/variance and overfitting.
17. ☐ Build a PyTorch Dataset/DataLoader.
18. ☐ Write a PyTorch training/validation loop.
19. ☐ Save and load a model checkpoint.
20. ☐ Diagnose an overfitting neural network.
21. ☐ Explain CNN and Transformer differences.
22. ☐ Explain Q/K/V attention in simple language.
23. ☐ Explain tokens and context windows.
24. ☐ Call an LLM API from Python.
25. ☐ Generate validated structured JSON output.
26. ☐ Implement streaming output.
27. ☐ Track token/cost/latency metrics.
28. ☐ Choose between two LLMs using an evaluation set.
29. ☐ Generate embeddings for documents and queries.
30. ☐ Implement cosine similarity.
31. ☐ Build a FAISS semantic search index.
32. ☐ Add metadata filtering to retrieval.
33. ☐ Build a basic RAG pipeline.
34. ☐ Improve RAG with hybrid search or reranking.
35. ☐ Add citations and a no-answer behavior.
36. ☐ Create an automated RAG evaluation dataset.

37. ☐ Trace an LLM/RAG request.
38. ☐ Build a tool-calling assistant.
39. ☐ Add state and retries to an agent workflow.
40. ☐ Protect an agent with tool allowlists and approvals.
41. ☐ Build a FastAPI inference endpoint.
42. ☐ Add validation, auth basics, logging and error handling.
43. ☐ Containerize the API with Docker.
44. ☐ Run a multi-service stack with Docker Compose.
45. ☐ Deploy a small service to one cloud platform.
46. ☐ Track an ML experiment and model version.
47. ☐ Set up basic CI checks with GitHub Actions.
48. ☐ Explain data/model drift and a monitoring plan.
49. ☐ Design a RAG system for production traffic.
50. ☐ Debug a failure involving API, DB, cache or model latency.
51. ☐ Explain your strongest project end-to-end in a 10-minute interview.

Score	Interpretation
0-30%	Beginner — continue foundations
31-50%	Learning — build more end-to-end projects
51-70%	Intermediate — strengthen production and interview depth
71-85%	Job Ready — start/continue applications and mock interviews
86-100%	Strong Fresher — focus on specialization, system design and stronger evidence

Final standard: do not wait until you know every framework. If you can build, test, deploy and explain 2-3 strong AI systems, solve Python/SQL problems, and discuss trade-offs, you are much closer to employability than someone who has only completed many courses.

31. ❏ What NOT to Learn Yet

- Do not train an LLM from scratch.
- Do not master all cloud providers.
- Do not learn every agent framework.
- Do not learn every vector database.
- Do not spend months on Kubernetes administration.
- Do not over-focus on GANs, legacy RNN architectures or classical NLP pipelines unless a target role needs them.
- Do not memorize framework APIs without understanding the underlying architecture.
- Do not build 20 toy chatbots; build a few evaluated, deployed systems.
- Do not add microservices, Kafka or Kubernetes just to make a portfolio project look complex.
- Do not skip testing, evaluation, security or documentation because the demo works.

Recommended learning loop

Learn concept → implement from scratch where useful → use library → compare result → build project → test → deploy → measure → explain in interview → refactor.

You are job-ready when you can demonstrate engineering judgment, not when your checklist reaches 100%.